

Drone Race Project Phase II Instructions

ESE 6510: Physical Intelligence

March 2026

Congrats on Phase 1! Now you will move onto Phase II and actually deploy your drone racing policies in the real world. Phase II is split into three stages, each a week-long: (i) Circle Track Zero-shot, (ii) Circle Track Optimizations, (iii) Powerloop Track to facilitate a smooth sim2real process.

- Circle Track Zero-shot: April 7th - 14th
- Circle Track Optimizations: April 14th - 21st
- Powerloop Track: April 21st - 28th

Each stage contains deliverables that you need to complete in order to progress. We will use the flying space in PERCH @ Pennovation to run our policies. On Ed, you will sign up for time-slots at Pennovation within the date ranges above.

Note that you **must only** come to Pennovation within your allotted time-slots and not whenever is convenient. PERCH has generously allowed us to use the space for our class and we must not abuse it. If you don't get the results you want during your allotted time-slot, you will have to wait for your next time-slot. Additionally you **must** provide a video in sim of the policy you intend to deploy in Pennovation. The TA present will review the video to ensure it is safe to deploy.

1 Circle Track Zero-shot

Deliverables:

- ☐ `controller_simple_policy.py`
- ☐ `policy.zip` (a .zip of your policy directory, containing the params folder as well as the .pt file)
- ☐ Plot comparing the zero-shot performance of Circle policy in sim and in real.

In Isaac Lab, you will train a policy for the Circle Track! The circle is a simple and safe track that we will first use to gauge how well our current training strategy transfers to the real world.

```
'circle': [  
    [ 0.0, 3.0, 0.75, 0.0, 0.0, 0.00],  
    [-1.5, 4.5, 0.75, 0.0, 0.0, -1.57],  
    [ 0.0, 6.0, 1.75, 0.0, 0.0, 3.14],  
    [ 1.5, 4.5, 0.75, 0.0, 0.0, 1.57]
```

In `quadcopter_env.py` in your Isaac Lab training codebase, navigate to the tracks dict (line 409 in the original class repo). Add the circle track to this dict and set `track_name` to `'circle'` (line 131 in the class repo) instead of the default `'powerloop'`. Now you can train your policy on the Circle Track.

You should train a Circle policy that is intentionally safe and slow; this will help you match the observation space in sim and real. Your training strategy is entirely up to you. One recommendation is reviewing the rewards that led to safe and smooth trajectories on the `powerloop` track in Phase 1. As you run your Circle Track policy in Pennovation, your data will hint you towards necessary changes in simulation.

Next, git clone our repo at `Jirl-upenn/ese651_sim2real`. This ROS2 codebase is used to deploy your drone racing policies. Our general pipeline is as follows: We receive motion capture estimates from the overhead Vicon cameras, transform the mocap estimates into an observation vector, feed the observations into your policies, then send the thrust

and body rate commands to the drone via radio. This codebase handles all of those things.

*Note that you will not be responsible for deploying this codebase– the TAs have already set up the infrastructure.

Since each team’s policy may have been trained with a different observation space and a different policy architecture, you will need to produce your own `controller_simple_policy.py` file from the `Jirl-upenn/ese651_sim2real` repo. In this file, you will define the architecture of your policy network as well as the observations. We have provided you with an example `get_observations()` method for your Isaac Lab environment that matches the observations in the existing `controller_simple_policy.py` file in the `Jirl-upenn/ese651_sim2real` repo. You can find this example method at `Files/Drone Race Project/ESE651-Vineet-get_observations`.

During a real flight, the system records several ROS2 topics into a bag file. We will focus on three of these ROS2 topics:

- `/odom` — Odometry from the motion capture system (ground truth state: position, orientation, velocity)
- `/ctbr_cmd` — Collective Thrust + Body Rate commands sent to the drone
- `/observations` — Gate observations (corner positions in body frame)

Once you arrive at Pennovation, you will deploy your Circle Track policy zero-shot. In the `Jirl-upenn/ese651_sim2real` repo, you will find a Python script (`process_bag_with_br_pos_export.py`) that reads a ROS2 bag file recorded during a real Crazyflie drone flight, extracts relevant state and command data, and generates useful plots with additional JSON exports. Feel free to modify this file as you’d like. These plots are critical to understand the drone’s behavior in the real world. You will then create a plot comparing the performance of your policy in sim and in real on the circle track as a deliverable.

In summary: You should arrive at Pennovation with your Circle Track policy trained in sim and your `controller_simple_policy` file and any necessary dependencies (or use the existing one and train your policy with the observations from `Files/Drone Race Project/ESE651-Vineet-get_observations`) The TAs will help you deploy your policy and collect data. **Remember to bring a laptop with VScode and a flash drive!**

1.1 Run

Run the script with:

```
python process_bag_with_br_pos_export.py <bag_path> <namespace> [t0] [tf]
```

`[t0]` and `[tf]` are optional start and end time args and `namespace` is the name of the Crazyflie used during the real-world deployment.

For example:

```
python process_bag_with_br_pos_export.py logs/Example_Circle_Traj crazy_jirl_b1
```

2 Circle Track Optimization

Deliverables:

- ☐ Circle policy must safely complete three laps in Pennovation.
- ☐ Plot comparing the updated performance of Circle policy in sim and in real.

Now for stage 2, you will refine your training strategy in Isaac Lab to improve the sim2real transfer on the Circle Track. Once again you will have a one-week window to come to Pennovation (within your reserved time-slots) and test+refine your policies. Based off of the lessons you learned from the prior stage, the most optimal way to approach this one-week window is to train multiple policies in advance that test different hypotheses. Perhaps you want to change the reward density/sparsity, reset strategies, domain randomization, simulator dynamics, etc. Everything is on the table!

Once your Circle Track policy can safely complete 3 laps in Pennovation, you can proceed with Stage 3.

3 Powerloop Track

Deliverables:

- Successful sim2real on the powerloop track.

You've made it to the final stage of the competition! At this point, you have a robust training routine in Isaac Lab that transfers smoothly to the Circle Track in Pennovation.

Now is your time to attempt the Powerloop Track at Pennovation! You will use this week to test your approach and make final optimizations to get a strong sim2real on the powerloop track. In order to qualify for the final race, you **must** be able to complete **3 laps** on the powerloop track during this week.

The final race will be on April 28th during class and at Pennovation.